

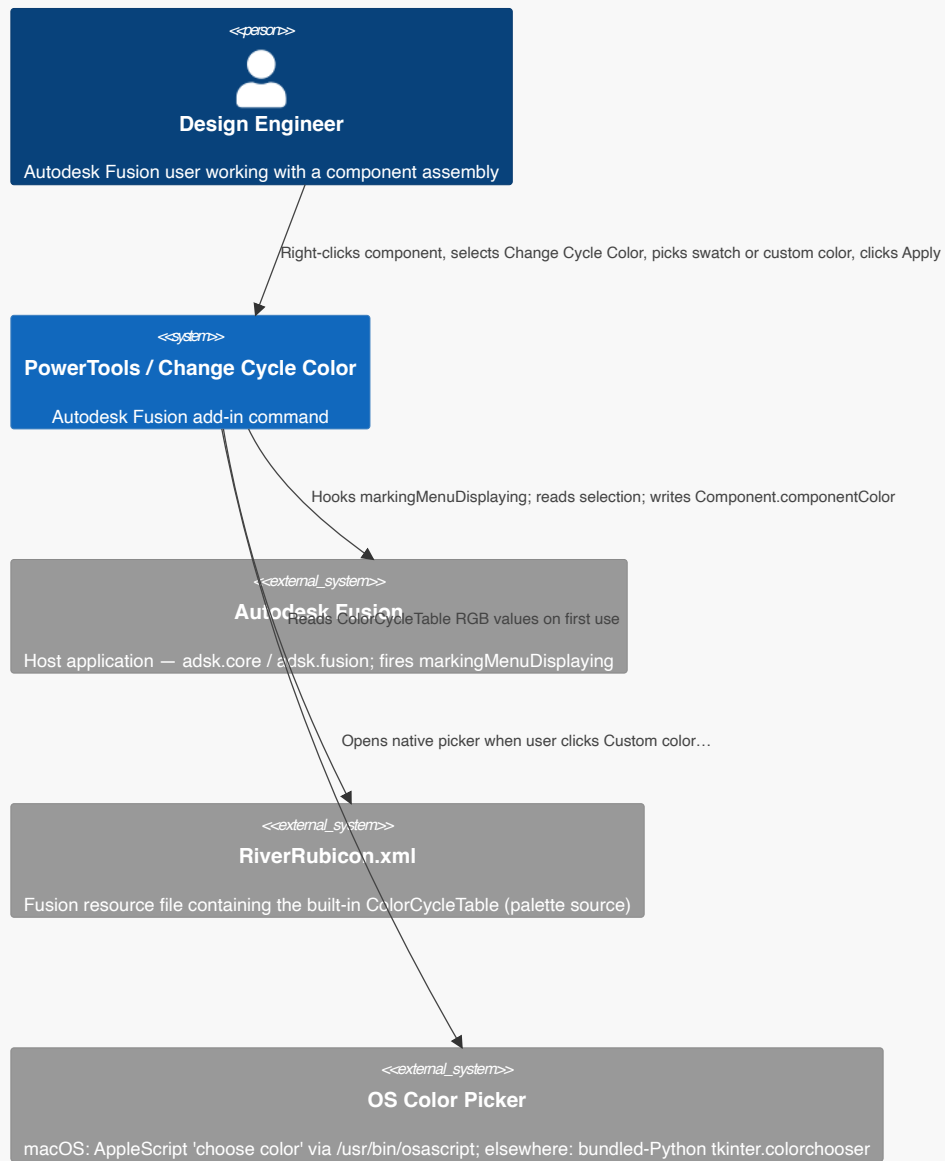
Change Cycle Color — Architecture

[← Change Cycle Color guide](#)

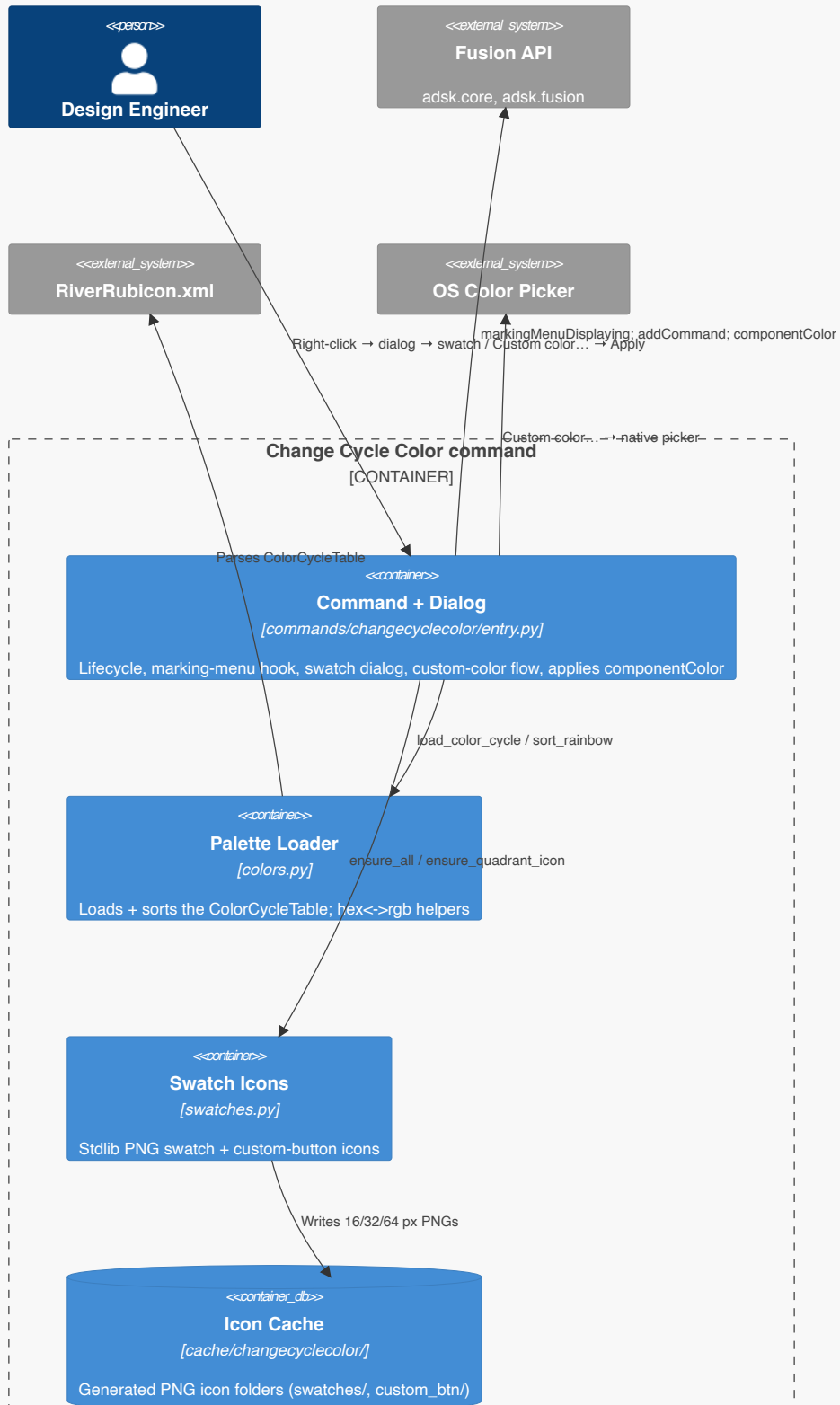
Architecture

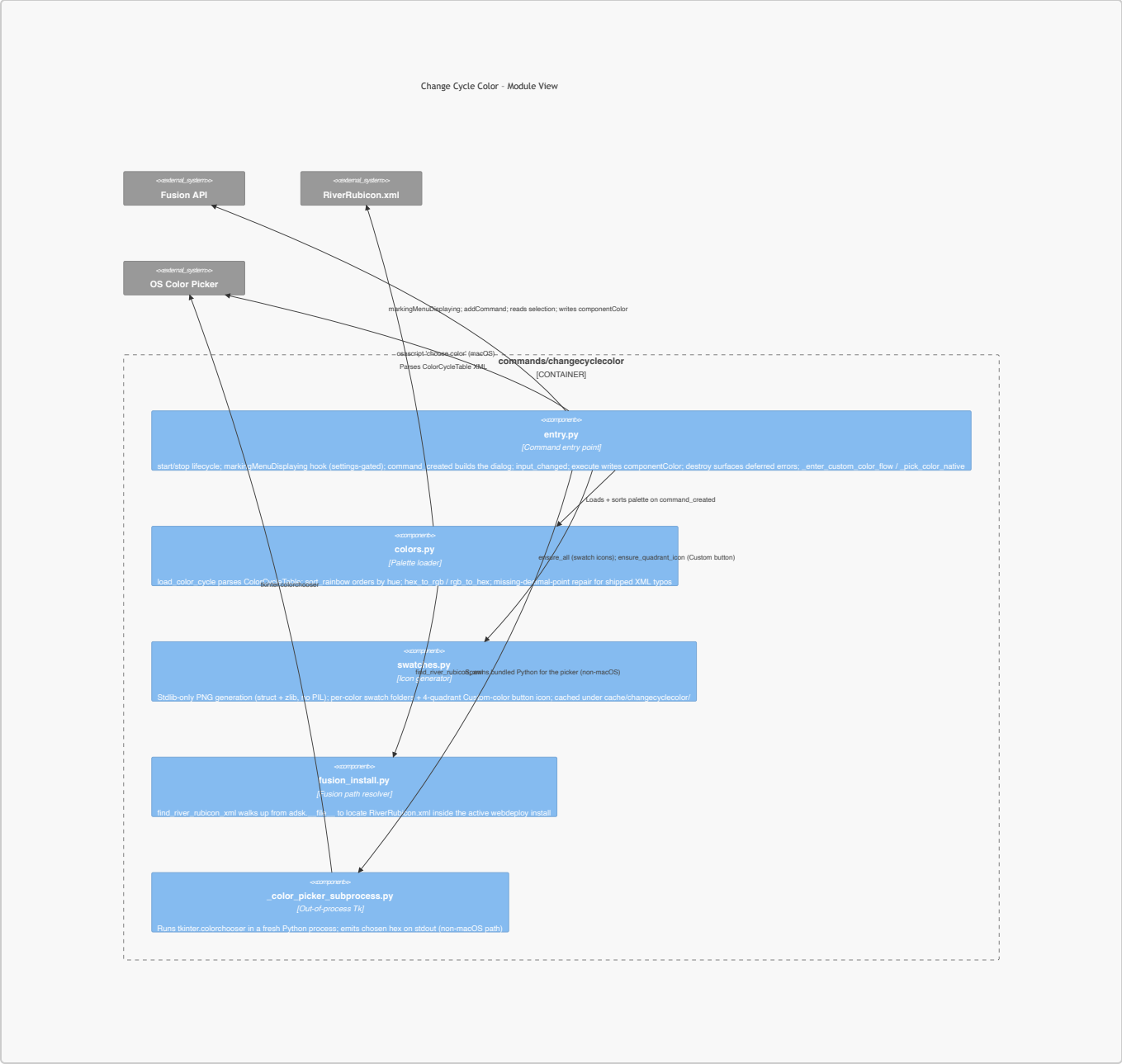
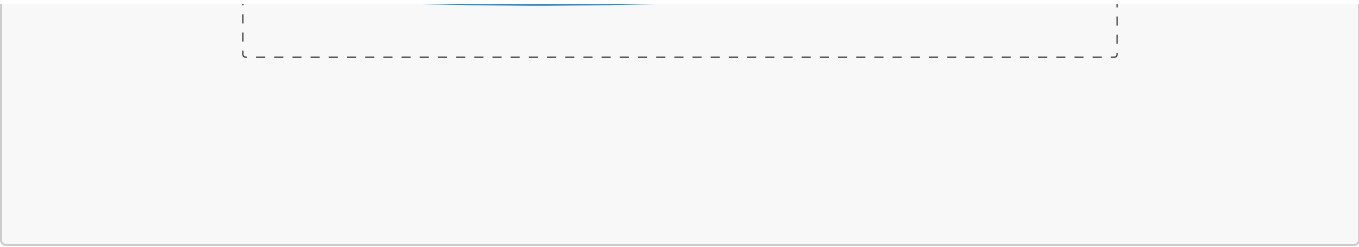
Change Cycle Color is a selection-driven command surfaced only through Fusion's right-click marking menu. It subscribes to the `markingMenuDisplaying` event and, when at least one Component or Occurrence is selected and its preference is enabled, injects its entry into the linear marking menu immediately after Fusion's built-in **Cycle Component Color** command. On invocation it opens a dialog containing rows of rainbow swatch buttons plus a **Custom color...** button. The palette is read live from Fusion's own `ColorCycleTable` in `RiverRubicon.xml` and rendered as cached PNG icons built with only the Python standard library. Applying a color writes `Component.componentColor` (never **Appearance**) through the Fusion Python API for every selected component. The custom-color path opens the OS-native picker, applies the color directly, and dismisses the dialog through Fusion's normal execute path — guarded by a `_skip_normal_execute` flag so the color is not applied twice.

Change Cycle Color - System Context



Change Cycle Color - Container View





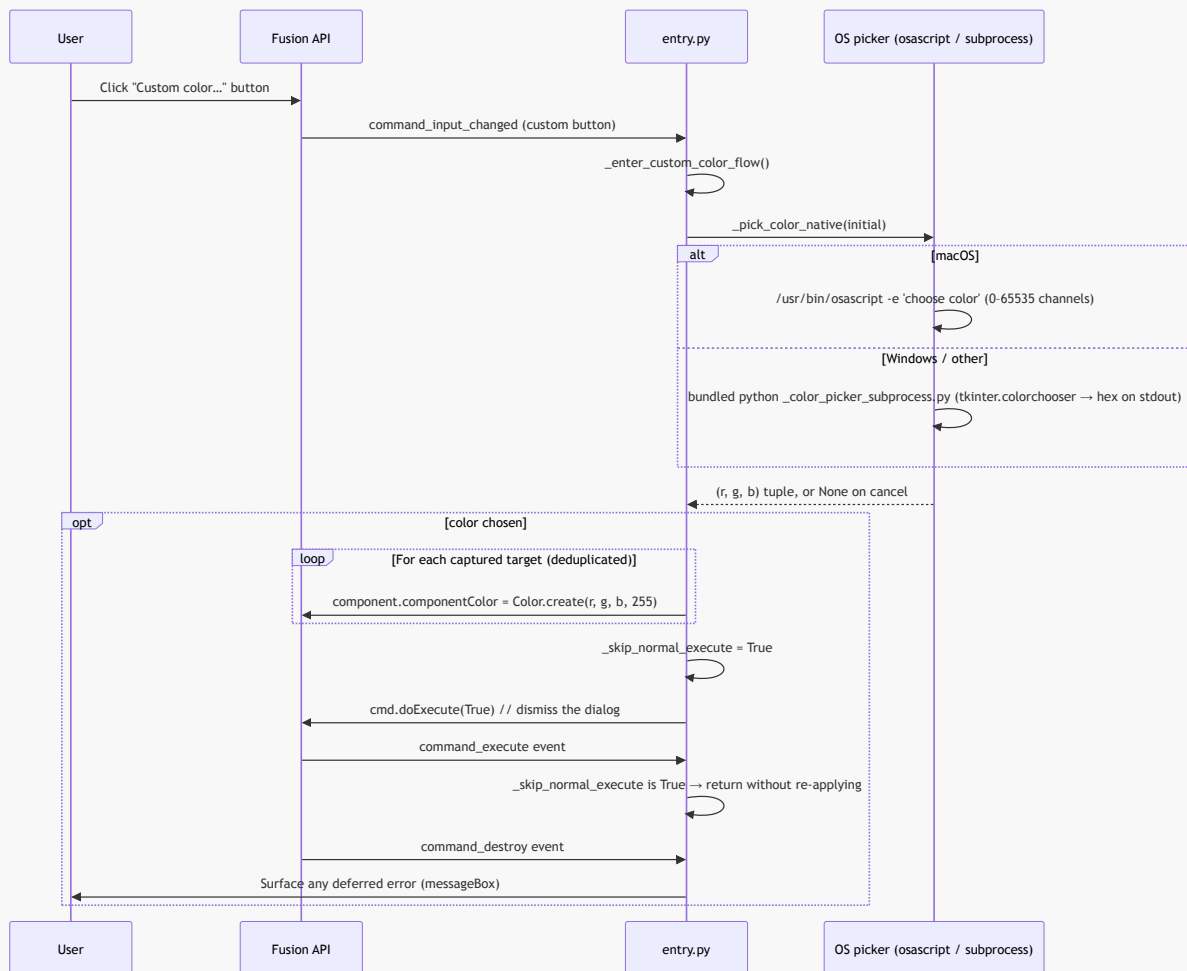
Main flow — swatch selection



Syntax error in text

mermaid version 11.16.0

Custom color flow



Module breakdown

- **entry.py** — command lifecycle (**start / stop**), the swatch dialog (**command_created**, **command_input_changed**, **command_execute**, **command_destroy**), the **markingMenuDisplaying** hook (**_on_marking_menu_displaying**, gated by the show-in-context-menu setting), selection collection (**_collect_selected_components**, instance dedupe), the custom-color flow (**_enter_custom_color_flow**, **_pick_color_native**,

`_pick_color_macos`, `_pick_color_subprocess_python`), and the apply step (`_set_component_color`, which writes `componentColor` only).

- **colors.py** — `load_color_cycle` parses the `ColorCycleTable` from `RiverRubicon.xml`; `sort_rainbow` orders swatches by hue (pushing pale neutrals to the end); `hex_to_rgb` / `rgb_to_hex` convert between formats. The RGB tokens are 0.0–1.0 floats, and the parser repairs shipped-XML typos where the leading decimal point is missing (e.g. "5412" → 0.5412).
- **swatches.py** — stdlib-only PNG generation (no PIL): `ensure_swatch_folder` / `ensure_all` write per-color 16/32/64 px solid swatch PNGs, and `ensure_quadrant_icon` writes the 4-quadrant rainbow icon for the **Custom color...** button. Icons are cached under `cache/changeccyclecolor/` (`swatches/` and `custom_btn/`) and regenerated only when missing.
- **fusion_install.py** — `find_river_rubicon_xml` discovers `RiverRubicon.xml` dynamically by walking up from `adsk.__file__` (falling back to `sys.executable`), so the path tracks Fusion `webdeploy` hash changes automatically.
- **_color_picker_subprocess.py** — a tiny standalone script that runs `tkinter.colorchooser` in a fresh Python process (used on non-macOS platforms), emitting the chosen hex on stdout. Running out-of-process avoids the in-process Tk run-loop conflict inside Fusion.

Integration into PowerTools

- Registered in `command_registry.py` under the **Assembly** group with `settings=True`, so it gets a PowerTools Preferences card.
- Started and stopped with the rest of the add-in by `commands/__init__.py` (`start()` / `stop()`).
- Uses the shared `lib/ptAddInUtils` helpers (`add_handler`, `log`, `handle_error`) for handler registration and logging.
- Reads its **Show in the right-click context menu** toggle through `settings_store.command_setting`; the `markingMenuDisplaying` handler re-reads it live, so the preference takes effect on the next right-click with no restart.

Design decisions

- **Stdlib-only PNG swatches.** Swatch and custom-button icons are built from raw bytes using only `struct` and `zlib`, producing valid 8-bit RGB PNGs. This avoids bundling PIL/Pillow into Fusion's embedded Python, which cannot reliably `pip install` extra dependencies.
- **Dynamic install-path discovery.** `RiverRubicon.xml` lives under a `webdeploy`-hash directory that changes with every Fusion update. Walking up from `adsk.__file__` (rather than hard-coding a path) keeps the palette pointed at the currently installed version and survives updates silently.
- **osascript on macOS (Gatekeeper workaround).** macOS Sequoia blocks Fusion from re-spawning its bundled `Python.app` GUI helper. `/usr/bin/osascript` is a system-signed binary at a fixed path that Gatekeeper always allows, and AppleScript's `choose color` uses `NSColorPanel` underneath — so macOS uses `osascript` while other platforms run `tkinter.colorchooser` in a subprocess.

- **Settings-gated, live context-menu entry.** The command surfaces only through the marking menu, gated by a preference the `markingMenuDisplaying` handler re-reads on every right-click. Turning the toggle off suppresses the entry immediately, with no handler re-registration and no Fusion restart.
- **`_skip_normal_execute` to prevent double-apply.** The custom-color flow applies the picked color directly, then calls `cmd.doExecute(True)` to close the dialog through Fusion's normal execute path. The flag tells `command_execute` that the work is already done so it returns without re-applying the (now stale) swatch selection.
- **Writes `componentColor` only.** The command sets `Component.componentColor` — the value the Color Cycling Toggle reads — and never touches **Appearance** or material, so it has no effect on rendering or physical properties.

[← Change Cycle Color guide](#)

Copyright © 2026 IMA LLC. All rights reserved.